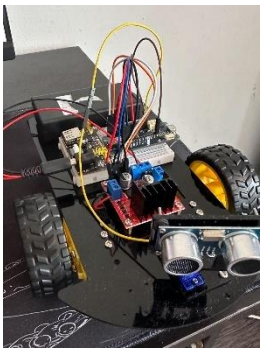
PROJETO MICROCONTROLADORES E MICROPROCESSADORES

RELATÓRIO FINAL



ROBÔ WIRELESS

Nome: Toni Correia

N.º: 126888

Curso: Desenvolvimento de Software

Disciplina: Microcontroladores e Microprocessador

RESUMO DO PROJETO

O projeto consiste no desenvolvimento de um robô móvel controlado remotamente por Bluetooth através da aplicação Dabble, utilizando um microcontrolador ESP32. O robô é equipado com dois motores DC controlados por uma ponte H e um sensor ultrassónico que permite evitar obstáculos, recuando automaticamente quando um objeto é detetado a curta distância.

ÍNDICE GERAL:

Índice

Resumo do Projeto 2

Índice Geral: 3

Índice de Figuras: 4

Finalidade do projeto: 5

Metodologia: 6

Programação. 6

Elaboração do esquema de ligação. 8

Realização de testes de funcionamento. 9

Montagem do robô. 9

Enquadramento teórico: 10

Requisitos: 11

Requisitos hardware: 11

Requisitos Software: 13

Realizações das concretizações do projeto: 14

Opções estratégicas: 14

Problemas e soluções encontradas: 15

Desenvolvimento futuro: 16

Reflexão final: 16

Webgrafia: 16

Anexo I - Código de testes 17

}Anexo II - Código Final 22

ÍNDICE DE FIGURAS:

Figura 1 – Teste de componentes 6

Figura 2 – Trecho do codigo 7

Figura 3 – Circuito Fritzing 8

Figura 4 - Testes 9

Figura 5 - Robô 9

Figura 6 – Kit robo 11

Figura 7 - Baterias 11

Figura 8 – L298N 12

Figura 9 - ESP32 12

Figura 10 - Fonte Breadboard 12

Figura 11 -Fritzing 13

Figura 12 – Arduino IDE 13

FINALIDADE DO PROJETO:

A principal finalidade do projeto é aplicar os conhecimentos adquiridos na área da eletrónica e programação de sistemas, através da construção de um veículo inteligente com controlo remoto e capacidade básica de reação ao ambiente. O objetivo é explorar conceitos de automação, controlo de motores, comunicação sem fios e integração de sensores.

METODOLOGIA:

O desenvolvimento do projeto seguiu uma abordagem prática baseada em:

Pesquisa dos componentes necessários para o braço.

Depois de ter visto inspirações de trabalhos eletrónicos e saber o que fazer, tive de pesquisar os componentes necessários para realizar o trabalho.

Elaboração do orçamento.

Ao saber os componentes necessários tive de fazer um orçamento para depois terem uma noção base do custo do projeto.

Teste dos componentes.

Tive de realizar os testes dos componetes para saber se estava tudo em conformidade para a realização do projeto.

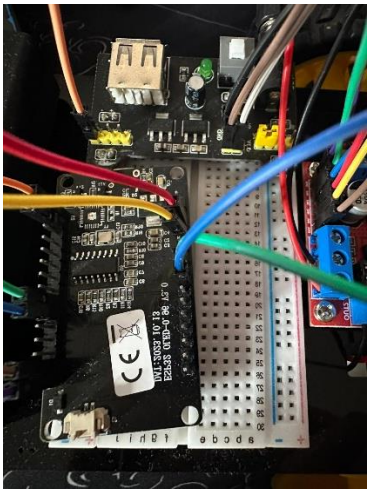


Figura 1 – Teste de componentes

Programação.

Programação do Esp 32 em Arduino IDE, depois de ter noção de como queria que o robô se comportasse foi só programar com a libraria DabbleEsp32 para comunicar com o aparelho.

ESP32_Bluetooth_Car.ino

```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include <DabbleESP32.h>

const int trigPin = 26;
const int echoPin = 25;

//Right motor
int enableRightMotor=22;
int rightMotorPin1=12;
int rightMotorPin2=13;
//Left motor
int enableLeftMotor=23;
int leftMotorPin1=18;
int leftMotorPin2=19;

#define MAX_MOTOR_SPEED 255
#define DIST_LIMIT_CM 10 // distância mínima para reagir

const int PWMFreq = 1000;
const int PWMResolution = 8;
const int rightMotorChannel = 4;
const int leftMotorChannel = 5;

void setupMotorPins() {
  pinMode(enableRightMotor, OUTPUT);
  pinMode(rightMotorPin1, OUTPUT);
  pinMode(rightMotorPin2, OUTPUT);

  pinMode(enableLeftMotor, OUTPUT);
  pinMode(leftMotorPin1, OUTPUT);
  pinMode(leftMotorPin2, OUTPUT);

  ledcSetup(rightMotorChannel, PWMFreq, PWMResolution);
  ledcSetup(leftMotorChannel, PWMFreq, PWMResolution);
```

Figura 2 – Trecho do codigo

Elaboração do esquema de ligação.

Criei o circuito no fritzing depois de já ter o código feito e a aplicação para começar os primeiros testes.

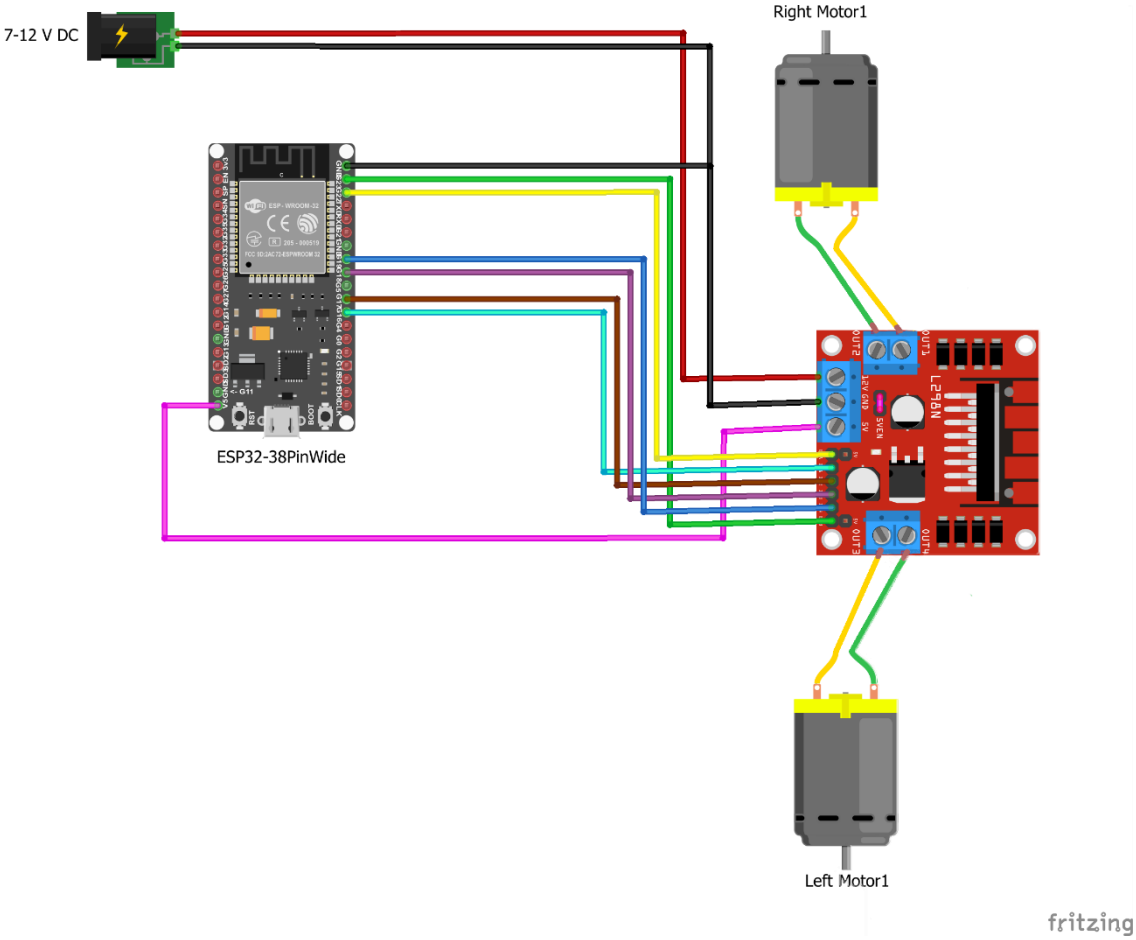


Figura 3 – Circuito Fritzing

Realização de testes de funcionamento.

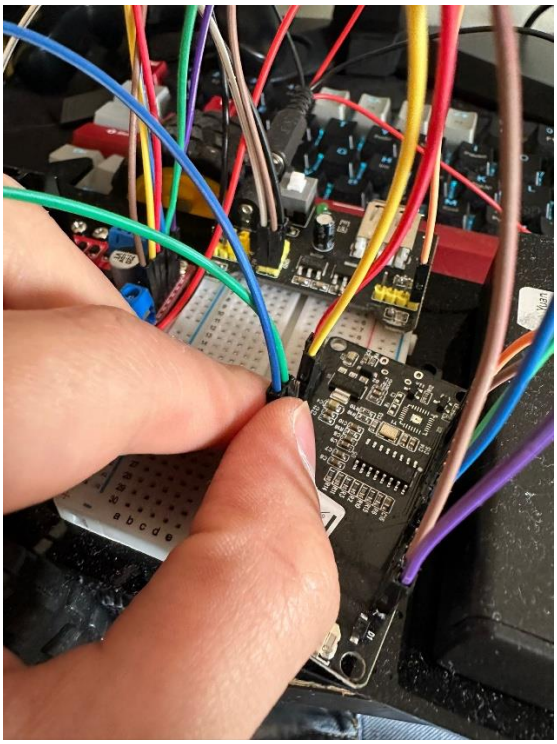


Figura 4 - Testes

Montagem do robô.

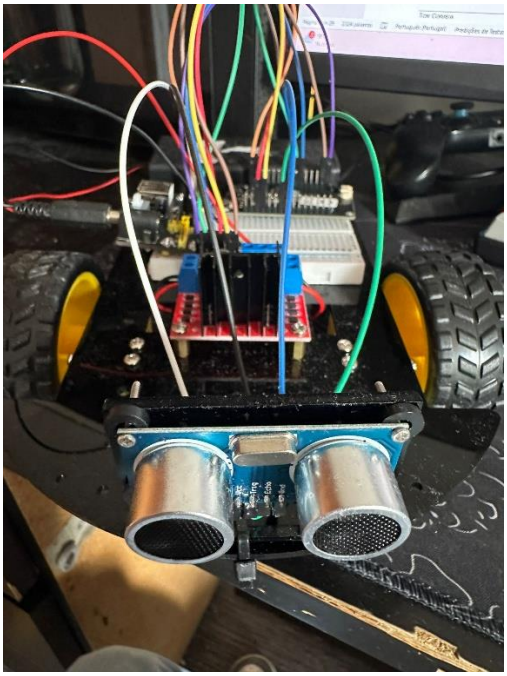


Figura 5 - Robô

ENQUADRAMENTO TEÓRICO:

O projeto baseia-se nos seguintes conceitos:

- Microcontroladores ESP32: sistema embebido com Wi-Fi/Bluetooth e suporte PWM.
- Motores DC e ponte H (L298N): controlo da direção e velocidade dos motores.
- Sensor ultrassónico (HC-SR04): cálculo da distância com base no tempo de eco.
- Comunicação Bluetooth com Dabble: interface entre smartphone e ESP32 via comandos digitais.
- PWM (modulação por largura de pulso): controlo da velocidade dos motores.

REQUISITOS:

Neste projeto necessito de três tipos de requisitos.

Requisitos hardware:

Quant	Designação	Preço unit(€)	Preço parcial(€)	Observações	Referência
1	Kit Robô	10.90	10.90	BotnRoll	ROB04005
2	18650 3,7V	4.60	9.20	BotnRoll	BAT03005
1	Driver Motores L298N	7.90	7.90	BotnRoll	MOT01063
1	Arduino ESP32	11.90	11.90	BotnRoll	PCO05149
1	Fonte de Alimentação para BreadBoard	9.20	9.20	BotnRoll	FAL01006
5	**Total**	44.5	49.1		

No meu projeto será utilizado:

Kit Robô

Com este **kit** podemos iniciar a construção do robô, necessitando apenas de acrescentar um controlador de motores, um Arduino, e baterias.

Figura 6 – Kit robo

Pilha Li-Ion 18650 3,7V 2600mAh

Bateria de lítio recarregável no formato 18650 (18mm diâmetro e 65mm comprimento) com 2600mAh de capacidade e capaz de debitar 1.3A (0.5C) continuamente e 7.8A (3C) de pico.

Figura 7 - Baterias

Ponte dupla H L298N

O driver de motor de ponte dupla H L298N é uma placa de controle de motor normalmente usada para controlar a velocidade e a direção de motores. Também pode ser usado para controlar o brilho de determinados projetos de iluminação, como conjuntos de LED de alta potência.

Figura 8 – L298N

Placa ESP32

Este ESP32-S3-DEV-KIT-N8R8 é uma placa de desenvolvimento compacta com várias interfaces.

Em termos de hardware, o módulo ESP32-S3-WROOM-1-N8R8 está equipado com um processador dual-core Xtensa 32-bit LX7 com uma frequência principal até 240MHz, 512KB SRAM (TCM) e 8MB PSRAM, e uma porta serie CH343 e CH334 USB HUB dual chip integrado na placa, permitindo assim o uso e desenvolvimento simultâneo de USB e UART através de uma interface USB-C.

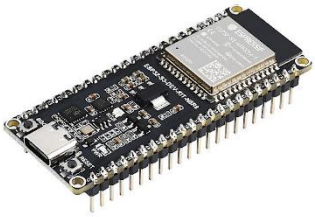


Figura 9 - ESP32

Fonte de alimentação para Breadboard

Com este conversor DC-DC pode alimentar com 5Vdc ou 3,3Vdc e 800mAh de corrente máxima as linhas de alimentação de uma breadboard standard. Para tal basta usar um cabo USB ou um transformador AC/DC com 5 a 12Vdc de saída.

Figura 10 - Fonte Breadboard

Requisitos Software:

Fritzing

Fritzing é um programa que permite planear as ligações antes de as fazer na prática, foi o programa utilizado para a criação do circuito**.**

Figura 11 -Fritzing

Arduino

O software que utilizei para escrever o código e enviá lo para o arduino o meter em prática foi o **Arduino Integrated Development Environment (IDE)** é um software open source e facilita a escrita de código e o upload para a placa. Este software pode ser usado em qualquer tipo de placa arduino.

Figura 12 – Arduino IDE

REALIZAÇÕES DAS CONCRETIZAÇÕES DO PROJETO:

- Montagem do circuito e ligações corretas entre os componentes.
- Implementação de controlo de direção e velocidade dos motores.
- Estabelecimento de comunicação Bluetooth com a aplicação Dabble.
- Leitura de distâncias pelo sensor ultrassónico.
- Lógica de recuo automático ao detetar obstáculos.
- Testes de campo com sucesso em ambiente de simulação real.

Opções estratégicas:

- Escolha do ESP32 em vez do Arduino Uno devido ao suporte nativo a Bluetooth.
- Utilização da app Dabble para evitar complicações com interfaces físicas.
- Integração de um sensor ultrassónico como primeira forma de “inteligência” do robô.
- Separação clara entre controlo manual e comportamento automático.

PROBLEMAS E SOLUÇÕES ENCONTRADAS:

Durante o desenvolvimento do projeto, surgiram algumas dificuldades técnicas que exigiram análise e adaptação para alcançar os objetivos pretendidos:

- Tensão de alimentação incompatível: O modelo do ESP32 utilizado apenas suporta alimentação de 3.3V, e não 5V como outros microcontroladores. Inicialmente, isso causou instabilidade no funcionamento, especialmente quando tentava alimentar o ESP32 diretamente da ponte H. Como solução, foi utilizada uma fonte de alimentação dedicada para breadboard, configurada para fornecer 3.3V de forma estável ao ESP32, permitindo o correto funcionamento de todo o sistema.
- Inversão na ligação dos motores: Ao testar os motores DC conectados à ponte H, foi identificado que os mesmos giravam no sentido oposto ao esperado. Isso ocorreu porque os terminais estavam trocados em relação à lógica do código. Para resolver a situação, foi necessário ajustar o código, invertendo os sinais enviados aos pinos IN1 e IN2 de cada motor, até que os movimentos correspondessem aos comandos recebidos pela aplicação Dabble.

Estas dificuldades, apesar de comuns em projetos com hardware, foram importantes para consolidar o entendimento sobre alimentação de circuitos, polaridade de motores e interpretação prática dos sinais de controlo.

DESENVOLVIMENTO FUTURO:

Com base nos resultados obtidos e nas aprendizagens adquiridas, existem várias possibilidades para expandir e melhorar o projeto no futuro:

- Integração de sensores adicionais: Adicionar sensores infravermelhos (IR) para deteção de linha ou prevenção de quedas, o que permitiria funcionalidades como seguir caminhos ou evitar degraus.
- Autonomia inteligente: Implementar algoritmos básicos de navegação autónoma, como desvio automático de obstáculos e mapeamento de trajetos, para permitir que o robô se mova de forma independente em ambientes desconhecidos.

REFLEXÃO FINAL:

O projeto permitiu consolidar conhecimentos teóricos e práticos sobre eletrónica, controlo de motores, sensores e comunicação sem fios. Mostrou-se um desafio completo, desde a montagem até à programação e resolução de problemas. Foi uma excelente experiência de aprendizagem e preparação para projetos mais complexos na área da robótica e automação.

WEBGRAFIA:

https://www.arduino.cc

https://fritzing.org

https://www.botnroll.com/pt/

ANEXO I - CÓDIGO DE TESTES

Teste servo motores

```
// Pinos dos motores
const int ENA = 22;
const int IN1 = 16;
const int IN2 = 17;
const int IN3 = 18;
const int IN4 = 19;
const int ENB = 23;

// Pinos do sensor ultrassónico
const int trigPin = 5;
const int echoPin = 4;

// PWM
const int motorSpeed = 200;  // 0 a 255

void setup() {
  // Inicialização dos pinos dos motores
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);

  // Sensor ultrassónico
```

```
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  Serial.begin(115200);
}

void loop() {
  float distance = getDistance();
  Serial.print("Distância: ");
  Serial.print(distance);
  Serial.println(" cm");

  if (distance > 0 && distance < 10) {
    // Muito perto, recuar
    andarTras();
    delay(800);
    parar();
  } else {
    // Movimento de teste: frente > parar > esquerda > parar > direita
    andarFrente();
    delay(1000);
    parar();
    delay(500);

    virarEsquerda();
    delay(1000);
    parar();
    delay(500);
```

```
    virarDireita();
    delay(1000);
    parar();
    delay(500);
  }

  delay(1000);
}

float getDistance() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duracao = pulseIn(echoPin, HIGH, 20000);  // Timeout 20 ms
  float distancia = duracao * 0.0343 / 2.0;

  if (duracao == 0) return -1;  // Sem leitura
  return distancia;
}

// Funções de controlo dos motores
void andarFrente() {
  digitalWrite(IN1, HIGH);
```

```
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, motorSpeed);
  analogWrite(ENB, motorSpeed);
}

void andarTras() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, motorSpeed);
  analogWrite(ENB, motorSpeed);
}

void virarEsquerda() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, motorSpeed);
  analogWrite(ENB, motorSpeed);
}

void virarDireita() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
```

```
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, motorSpeed);
  analogWrite(ENB, motorSpeed);
}

void parar() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
```

}ANEXO II - CÓDIGO FINAL

Controlo de braço robótico.

```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include <DabbleESP32.h>

const int trigPin = 26;
const int echoPin = 25;

//Right motor
int enableRightMotor=22;
int rightMotorPin1=12;
int rightMotorPin2=13;
//Left motor
int enableLeftMotor=23;
int leftMotorPin1=18;
int leftMotorPin2=19;

#define MAX_MOTOR_SPEED 255
#define DIST_LIMIT_CM 10 // distância mínima para reagir

const int PWMFreq = 1000;
const int PWMResolution = 8;
const int rightMotorChannel = 4;
const int leftMotorChannel = 5;
```

```
void setupMotorPins() {
  pinMode(enableRightMotor, OUTPUT);
  pinMode(rightMotorPin1, OUTPUT);
  pinMode(rightMotorPin2, OUTPUT);

  pinMode(enableLeftMotor, OUTPUT);
  pinMode(leftMotorPin1, OUTPUT);
  pinMode(leftMotorPin2, OUTPUT);

  ledcSetup(rightMotorChannel, PWMFreq, PWMResolution);
  ledcSetup(leftMotorChannel, PWMFreq, PWMResolution);
  ledcAttachPin(enableRightMotor, rightMotorChannel);
  ledcAttachPin(enableLeftMotor, leftMotorChannel);

  controlMotors(0, 0);
}

void controlMotors(int rightSpeed, int leftSpeed) {
  // Motor direito
  digitalWrite(rightMotorPin1, rightSpeed > 0);
  digitalWrite(rightMotorPin2, rightSpeed < 0);

  // Motor esquerdo
  digitalWrite(leftMotorPin1, leftSpeed > 0);
  digitalWrite(leftMotorPin2, leftSpeed < 0);
```

```
  ledcWrite(rightMotorChannel, abs(rightSpeed));
  ledcWrite(leftMotorChannel, abs(leftSpeed));
}

float getDistanceCM() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 20000); // timeout: 20 ms
  float distance = duration * 0.0343 / 2.0;
  return distance;
}

void setup() {
  setupMotorPins();

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  Dabble.begin("CARROdoTONI");
}
```

```
void loop() {
  float distance = getDistanceCM();
  Dabble.processInput();

  if (distance > 0 && distance < DIST_LIMIT_CM) {
    // Muito perto, recuar
    controlMotors(-MAX_MOTOR_SPEED, MAX_MOTOR_SPEED);
    delay(500); // anda para trás 0.5 s
    controlMotors(0, 0);
    return;
  }

  // Controlo manual via Gamepad
  int rightSpeed = 0;
  int leftSpeed = 0;

  if (GamePad.isDownPressed()) {
    rightSpeed = -MAX_MOTOR_SPEED;
    leftSpeed = MAX_MOTOR_SPEED;
  }
  if (GamePad.isUpPressed()) {
    rightSpeed = MAX_MOTOR_SPEED;
    leftSpeed = -MAX_MOTOR_SPEED;
  }
  if (GamePad.isRightPressed()) {
    rightSpeed = MAX_MOTOR_SPEED;
```

```
    leftSpeed = MAX_MOTOR_SPEED;
  }
  if (GamePad.isLeftPressed()) {
    rightSpeed = -MAX_MOTOR_SPEED;
    leftSpeed = -MAX_MOTOR_SPEED;
  }

  controlMotors(rightSpeed, leftSpeed);
}
```